Accuracy, Precision, Recall (ML Evaluation Metrics)

These are **metrics used to evaluate the performance of a machine learning model**, especially in classification problems.

Used widely with libraries like Scikit-learn.

1. Accuracy

Accuracy = Number of correct predictions out of total predictions

Formula:
Accuracy = Correct Predictions/Total PredictionsAccuracy

Meaning:

"How often the model is correct overall"

2. Precision

Precision = Out of predicted positive cases, how many are actually positive

Formula:
Precision=TP/(TP+FP)

Where:

- TP = True Positive
- FP = False Positive

Meaning:

"When the model predicts YES, how often is it correct?"

3. Recall

Recall = Out of actual positive cases, how many were correctly predicted

Formula:
Recall=TP/(TP+FN)

Where:

- FN = False Negative

Meaning:

"Out of all real YES cases, how many did the model catch?"

4. F1 Score

F1 Score is the **balance between Precision and Recall**.

Simple:
"F1 Score tells how good the model is when we consider both Precision and Recall together."

Formula

F1 Score = 2×Precision×Recall / (Precision+Recall)

Meaning

- Combines **Precision + Recall**
- Useful when:
    - Data is **imbalanced**
    - Both false positives & false negatives matter

Biotech Example (Disease Detection)

Suppose you are building a model to detect a disease (e.g., cancer).

Dataset:

- Total patients = 100
- Actually diseased = 20
- Healthy = 80

Model Predictions:

- True Positive (TP) = 15 → correctly detected disease
- False Positive (FP) = 10 → wrongly predicted disease
- False Negative (FN) = 5 → missed disease
- True Negative (TN) = 70 → correctly predicted healthy

Calculations

1. Accuracy:

Correct predictions = TP + TN = 15 + 70 = 85

Accuracy = 85 / 100 = **85%**

2. Precision:

Precision = 15 / (15 + 10)
= 15 / 25 = **60%**

3. Recall:

Recall = 15 / (15 + 5)
= 15 / 20 = **75%**

4. Interpretation

Accuracy = 85%

The model looks good overall

Precision = 60%

Problem:

- 40% of predicted disease cases are wrong
  (Too many false alarms)

Recall = 75%

Good:

- Model detects 75% of actual patients
  (Still misses 25% — risky)

Small Dataset Example (Easy to Understand)

Problem

Predict whether an email is **Spam (1)** or **Not Spam (0)**

Dataset

Actual values (Real):

```
y_true = [1, 0, 1, 0, 1, 0]
```

Predicted values (Model output):

```
y_pred = [1, 0, 0, 0, 1, 1]
```

Step-by-Step Understanding

Index	Actual (y_true)	Predicted (y_pred)	Result
1	1	1	TP
2	0	0	TN
3	1	0	FN
4	0	0	TN
5	1	1	TP
6	0	1	FP

Count Values

- TP = 2

- TN = 2
- FP = 1
- FN = 1

Python Code (Scikit-learn)

```
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix

# Actual and Predicted values
y_true = [1, 0, 1, 0, 1, 0]
y_pred = [1, 0, 0, 0, 1, 1]

# Metrics
accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred)
recall = recall_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)

# Confusion Matrix
cm = confusion_matrix(y_true, y_pred)

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
print("Confusion Matrix:\n", cm)
```

Output

```
Accuracy: 0.66
Precision: 0.66
Recall: 0.66
F1 Score: 0.66
Confusion Matrix:
[[2 1]
 [1 2]]
```

Simple Explanation

- **Accuracy = 66%** → Model correct 4 out of 6 times
- **Precision = 66%** → Out of predicted spam, 66% correct
- **Recall = 66%** → Out of real spam, 66% detected
- **F1 Score = 66%** → Balanced performance

MAE, MSE, RMSE (Clean Notes – Copy Friendly)

1. Mean Absolute Error (MAE)

Formula:

```
MAE = (1/n) * Σ |yi - y_pred|
```

Meaning:

- Average of absolute differences between actual and predicted values
- No negative values (absolute is used)

Example:

```
y_true = [10, 20, 30]
y_pred = [12, 18, 33]

Errors = |10-12| + |20-18| + |30-33|
       = 2 + 2 + 3 = 7

MAE = 7 / 3 = 2.33
```

Key Points:

- Easy to understand
- Not heavily affected by outliers

2. Mean Squared Error (MSE)

Formula:

```
MSE = (1/n) * Σ (yi - y_pred)^2
```

Meaning:

- Squares the error before averaging
- Large errors become much bigger

Example:

```
Errors = (2^2 + 2^2 + 3^2)
       = 4 + 4 + 9 = 17

MSE = 17 / 3 = 5.67
```

Key Points:

- Penalizes large errors
- Commonly used in training models

3. Root Mean Squared Error (RMSE)

Formula:

```
RMSE = sqrt( (1/n) * Σ (yi - y_pred)^2 )
```

Meaning:

- Square root of MSE
- Output is in same unit as original data

Example:

```
RMSE = sqrt(5.67) = 2.38
```

Key Points:

- Easy to interpret
- Most commonly used metric

Comparison Table

```
Metric   | Outlier Impact | Unit        | Easy to Understand
---------|----------------|-------------|--------------------
MAE      | Low            | Same        | Yes
MSE      | High           | Squared     | No
RMSE     | High           | Same        | Yes
```

Python Code (Copyable)

```
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

y_true = [10, 20, 30]
y_pred = [12, 18, 33]

mae = mean_absolute_error(y_true, y_pred)
mse = mean_squared_error(y_true, y_pred)
rmse = np.sqrt(mse)

print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
```

Final Understanding

```
MAE  → average error
MSE  → focuses more on large errors
RMSE → balanced + practical (most used)
```

Project Notes: Disease Prediction using Machine Learning

1. Project Title: **Disease Prediction using Patient Health Data**

2. Objective

The goal of this project is to build a Machine Learning model that can predict whether a patient has a disease or not based on medical data.

3. Problem Type

- Type: **Classification Problem**
- Output:
  - 0 → No Disease
  - 1 → Disease

4. Dataset Description

The dataset contains medical attributes of patients.

Features (Independent Variables - X):

- Pregnancies
- Glucose Level
- Blood Pressure
- Skin Thickness
- Insulin
- BMI (Body Mass Index)
- Age

Target (Dependent Variable - y):

- Outcome (0 or 1)

5. Algorithm Used

Logistic Regression

- Used for classification problems
- Predicts probability of a class (0 or 1)

6. Steps Involved in the Project

Step 1: Import Libraries

- pandas → data handling
- sklearn → machine learning tools

Step 2: Load Dataset

- Read dataset using pandas
- Check structure using `.head()`

Step 3: Data Preprocessing

- Handle missing values (if any)
- Separate features and target

Step 4: Train-Test Split

- Split dataset into training and testing sets
- Common ratio: 80% training, 20% testing

Step 5: Feature Scaling

- Apply StandardScaler
- Ensures all features are on same scale

Step 6: Model Training

- Train Logistic Regression model using training data

Step 7: Prediction

- Predict results using test data

Step 8: Model Evaluation

- Accuracy Score → overall performance
- Confusion Matrix → detailed performance

7. Python Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
import numpy as np

# Load dataset
df = pd.read_csv("diabetes.csv")

# Features and Target
X = df.drop("Outcome", axis=1)
y = df["Outcome"]

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Feature Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Model Training
model = LogisticRegression()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

8. Output (Example)

```
Accuracy: 0.78

Confusion Matrix:
[[80 10]
 [20 44]]
```

9. Applications

- Medical diagnosis systems
- Early disease detection
- Healthcare analytics
- Clinical decision support

10. Advantages

- Fast and simple model
- Easy to interpret
- Works well for binary classification

11. Limitations

- Assumes linear relationship
- Not suitable for very complex data

12. Future Improvements

- Use advanced models (Decision Tree, Random Forest)
- Add more features
- Deploy as a web application (Flask)
- Improve accuracy using hyperparameter tuning

Unsupervised Learning

Unsupervised Learning is a type of machine learning where the model learns patterns from data **without labeled outputs**.

Simple Meaning:

Learning without a teacher.
The model finds hidden patterns or groups in data.

Example:

Customer data:

Age | Spending
20 | 2000
22 | 2500
45 | 8000

Model automatically groups:

- Young low spenders
- Older high spenders

Where it is used:

- Customer segmentation
- Gene analysis (biotech)
- Recommendation systems
- Image grouping

2. Mean (Average)

Mean = sum of all values / total number of values

Example:

Values: 2, 4, 6

Mean = (2 + 4 + 6) / 3 = 4

Use in ML:

Used to find the **center of data**

3. Centroid

Centroid is the **center point of a cluster** (average position of all points)

Example:

Points: (2,2) and (4,4)

Centroid = ((2+4)/2 , (2+4)/2) = (3,3)

Use:

Used in clustering algorithms like K-Means

4. Distance

Distance tells how far two points are from each other

Formula (text):

Distance = sqrt((x2 - x1)^2 + (y2 - y1)^2)

Example:

Points: (1,2) and (4,6)

Distance = sqrt((4-1)^2 + (6-2)^2)
= sqrt(9 + 16)
= sqrt(25) = 5

Use:

Used to decide which cluster a point belongs to

5. Variance

Variance measures how far values are spread from the mean

Formula (text):

Variance = (sum of (xi - mean)^2) / n

Example:

Values: 2, 4, 6
Mean = 4

Variance = ((2-4)^2 + (4-4)^2 + (6-4)^2) / 3
= (4 + 0 + 4) / 3
= 2.67

Use:

Used to measure data spread

6. Spread

Spread means how widely data is distributed

Example:

Data A: 4, 5, 6 → small spread
Data B: 1, 10, 20 → large spread

Related Concepts:

- Variance
- Standard deviation

---

7. Basic Algebra Concepts in ML

1. Linear Equation

Form:
y = mx + c

Where:

- m = slope
- c = intercept

Example:

y = 2x + 3

If x = 5 → y = 13

2. Squaring Values

Used in:

- Distance calculation
- Variance

Example:
(5 - 3)^2 = 4

3. Summation (Sigma)

Symbol: Σ

Meaning: Add all values

Example:
Σx = 2 + 4 + 6 = 12

Complete Example (Clustering)

Problem: Group students based on marks

Data points:
(10, 20), (12, 22), (50, 60), (55, 65)

Step 1: Create Groups

Group 1:
(10,20), (12,22)

Group 2:
(50,60), (55,65)

Step 2: Find Centroid

Group 1 centroid:
(11, 21)

Group 2 centroid:
(52.5, 62.5)

Step 3: Assign New Point

New point: (13, 23)

Check distance from both centroids
Assign to nearest group

Step 4: Check Spread

Group 1 → small spread
Group 2 → small spread

Clustering is good

Final Summary

- Unsupervised Learning → no labels, find patterns
- Mean → average value
- Centroid → center of cluster
- Distance → how far points are
- Variance → how spread data is
- Spread → how scattered data is
- Algebra → helps in calculations

What is K-Means?

K-Means is an unsupervised learning algorithm used to divide data into K number of clusters (groups).

Simple Meaning:

"Group similar data points together"

How K-Means Works (Step-by-Step)

1. Choose number of clusters (K)
2. Randomly select K centroids (center points)
3. Assign each data point to the nearest centroid (using distance)
4. Recalculate centroid (mean of points in cluster)
5. Repeat until clusters stop changing

Example (Simple Understanding)

Problem:

Group students based on marks

Data:
(10,20), (12,22), (50,60), (55,65)

Step 1: Choose K = 2

We want 2 groups

Step 2: Initial Centroids (random)

Suppose:
C1 = (10,20)
C2 = (50,60)

Step 3: Assign Points

Points near C1 → Group 1
Points near C2 → Group 2

Step 4: Recalculate Centroids

Group 1 → (10,20), (12,22)
New centroid = (11,21)

Group 2 → (50,60), (55,65)
New centroid = (52.5, 62.5)

Step 5: Repeat

Continue until no change

What Problem Does K-Means Solve?

It solves **grouping problems**

Real-life uses:

- Customer segmentation
- Student performance grouping
- Gene grouping (biotech)
- Image compression
- Market analysis

What is Elbow Method?

The **Elbow Method is used to find the best value of K (number of clusters).**

Idea:

- Try different K values (1,2,3,4…)
- Calculate error (inertia)
- Plot graph

Where graph bends like an "elbow" → best K

Simple Meaning:

"Find optimal number of clusters"

Example:

K = 1 → High error
K = 2 → Less error
K = 3 → Slight improvement

Elbow at K = 2

Code Example (K-Means)

You already saw the graph generated above
Here is the **clean code for your notes**:

```
import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Sample data
X = np.array([
    [1, 2], [2, 3], [3, 3], [2, 2],
    [8, 7], [9, 8], [10, 8], [9, 7]
])

# Apply KMeans
kmeans = KMeans(n_clusters=2, random_state=0)
kmeans.fit(X)

# Get labels and centroids
labels = kmeans.labels_
centroids = kmeans.cluster_centers_

# Plot data
plt.scatter(X[:, 0], X[:, 1])
plt.scatter(centroids[:, 0], centroids[:, 1], marker='x')
plt.title("K-Means Clustering")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
```

What You Saw in Graph

- Data automatically divided into **2 groups**
- 'X' marks = centroids
- Points closer to each other form clusters

Final Summary

- **K-Means** → Groups similar data
- **Centroid** → Center of cluster
- **Distance** → Used to assign points
- **Elbow Method** → Finds best K
- Works without labels (unsupervised learning)

What is PCA?

PCA (Principal Component Analysis) is a technique used to reduce the number of features (columns) while keeping maximum important information.

Simple Meaning:

"Reduce columns but keep useful information"

Why PCA is Used?

- Too many columns → complex model
- Slower training
- More memory usage

PCA solves this by:
Reducing dimensions
Keeping important patterns

Example (Concept)

Problem:

You have 4 columns:

| Height | Weight | Age | Salary |

PCA converts this into:

| PC1 | PC2 |

Now:

- 4 columns → 2 columns
- Less complexity
- Faster model

How PCA Works (Simple Idea)

1. Finds directions with maximum variation
2. Creates new features (PC1, PC2, …)
3. Keeps most important ones

Important Terms

- **Principal Component (PC)** → new feature
- **PC1** → most important
- **PC2** → second important

Example (Reduce 4 Columns → 2 Columns)

Sample Data

```
import pandas as pd

data = {
    "Height": [150, 160, 170, 180],
    "Weight": [50, 60, 70, 80],
    "Age": [20, 25, 30, 35],
    "Salary": [20000, 30000, 40000, 50000]
}

df = pd.DataFrame(data)
print(df)
```

Apply PCA (Scikit-learn)

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# Step 1: Scale data
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df)

# Step 2: Apply PCA (reduce to 2 columns)
pca = PCA(n_components=2)
reduced_data = pca.fit_transform(scaled_data)

print(reduced_data)
```

Important Points

- PCA creates **new features (not original ones)**
- It removes **less important information**
- Works best after **feature scaling**

When to Use PCA?

- Too many columns (high dimensional data)
- Visualization (2D/3D plotting)
- Remove noise
- Speed up models

Project: Customer Clustering using K-Means

Step 1: Problem Statement

We want to **group customers into clusters** based on:

- Age
- Income
- Spending

Goal:
Find similar customers
Create clusters (groups)

Step 2: Raw Data (Dirty Data)

```
import pandas as pd

data = {
    "Age": [25, 30, None, 28, 35, 40, None, 38],
    "Income": [30000, 40000, 50000, None, 60000, 65000, 70000, None],
    "Spending": [200, 250, None, 220, 300, 320, 330, None]
}

df = pd.DataFrame(data)
print(df)
```

Data has **missing values**

Step 3: Data Cleaning

Fill missing values:

- Age → Mean
- Income → Median
- Spending → Mean

```
df["Age"] = df["Age"].fillna(df["Age"].mean())
df["Income"] = df["Income"].fillna(df["Income"].median())
df["Spending"] = df["Spending"].fillna(df["Spending"].mean())
```

Now data is **clean**

Step 4: Feature Scaling

Important for K-Means

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaled_data = scaler.fit_transform(df)
```

Step 5: Apply K-Means Model

```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=2, random_state=0)
labels = kmeans.fit_predict(scaled_data)

df["Cluster"] = labels
print(df)
```

Now each customer has a **cluster (0 or 1)**

Step 6: Visualization (Graph)

You already saw the graph above

Code for your notes:

```
import matplotlib.pyplot as plt

plt.scatter(df["Age"], df["Income"])
plt.title("Customer Clustering using K-Means")
plt.xlabel("Age")
plt.ylabel("Income")
plt.show()
```

What Happened in This Project?

1. Took raw data
2. Cleaned missing values
3. Scaled data
4. Applied K-Means
5. Created clusters

6. Visualized result

Real-Life Use of This Project

- Marketing strategy
- Customer targeting
- Sales analysis
- Business segmentation

Important Learning

- Data cleaning is very important
- Scaling improves clustering
- K-Means groups similar data
- Visualization helps understanding